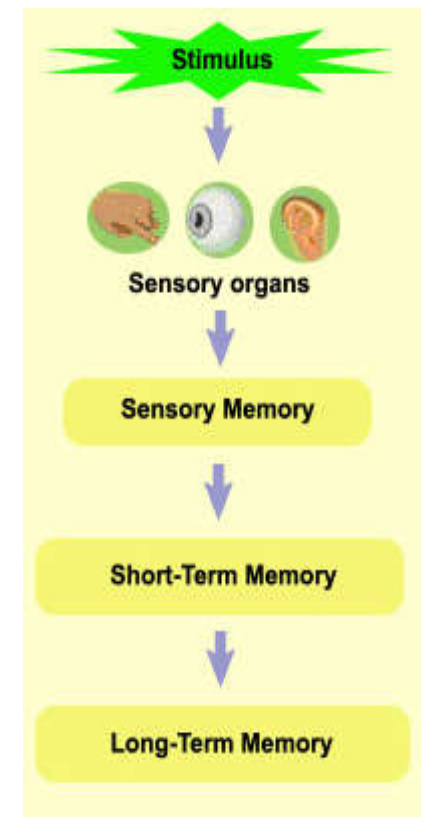
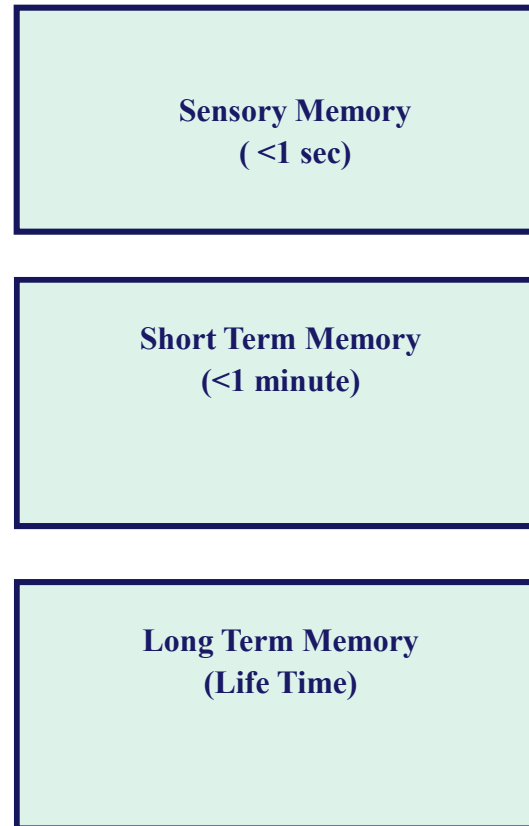


Data types, variables, constants

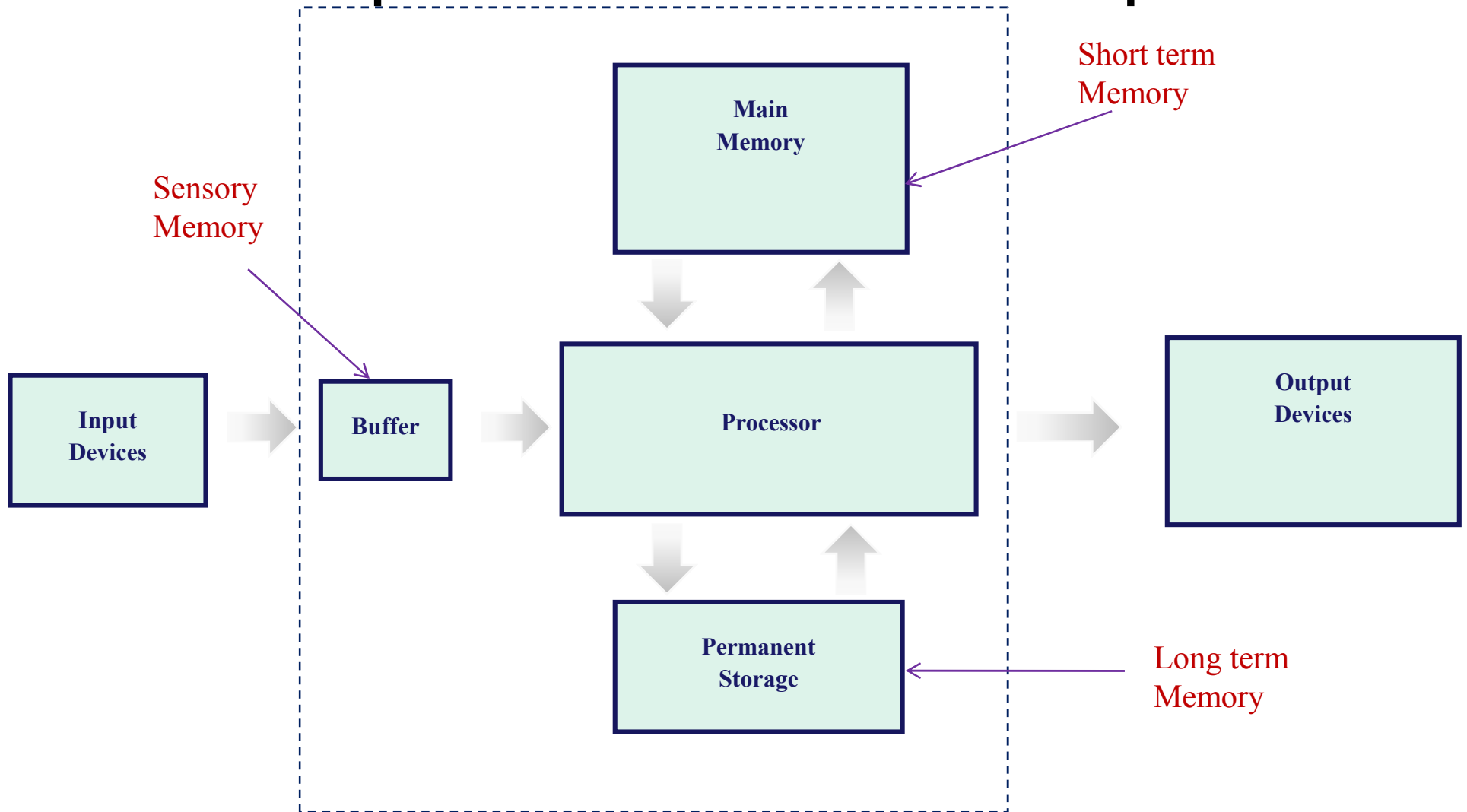
Outline

- 2.1 Introduction
- 2.2 A Simple C Program: Printing a Line of Text
- 2.3 Memory Concepts
- 2.4 Naming Convention of Variables
- 2.5 Arithmetic in C
- 2.6 Type Conversion

Computer was modelled after Human



A simple model of the computer

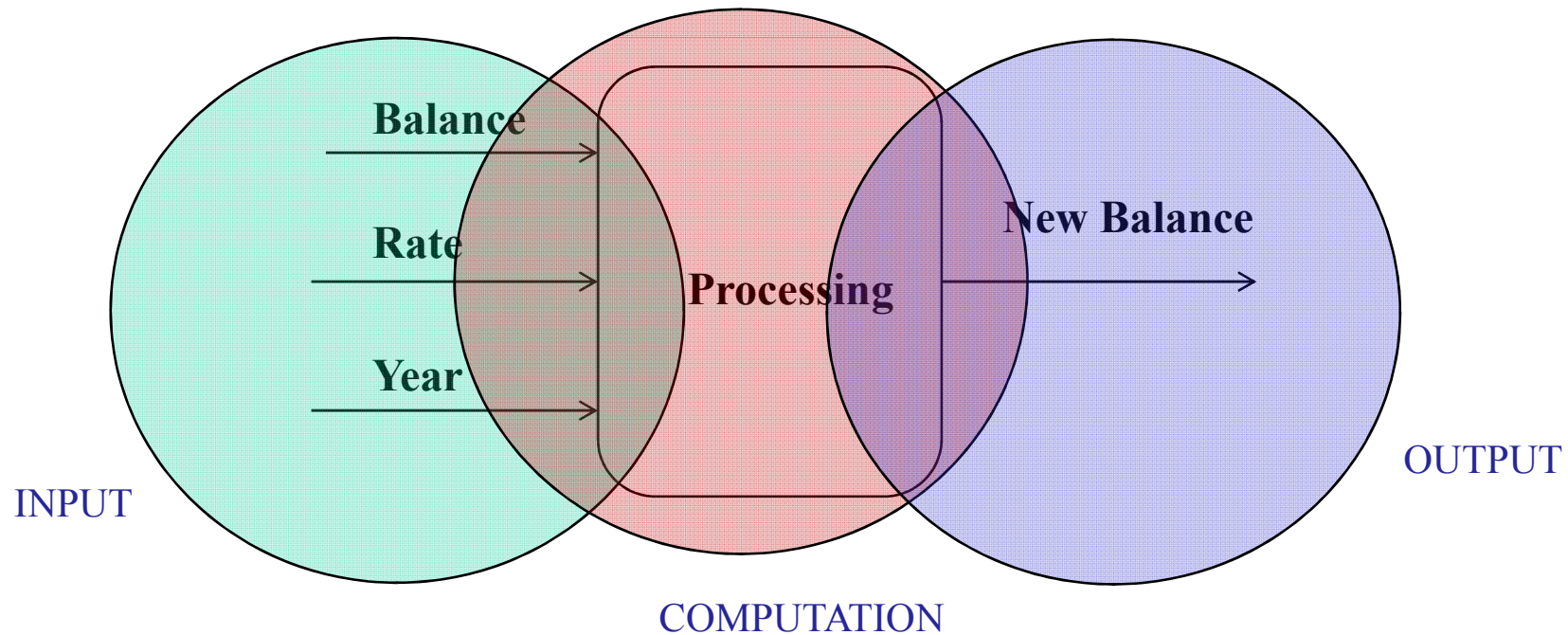


Definition: Computer Program

A Computer program is a sequence of instructions written to perform a specified task with a computer.

Example: Computer Program

Computer tell me what would be my balance after 1 year if my starting balance is 1,00,000 and interest rate is 10%.



Introduction

- C programming language
 - Structured and disciplined approach to program design
 - Story → Paragraph → Sentence
 - Program → Function → Statement

A Simple C Program: Printing a Line of Text

```
1  /* Fig. 2.1: fig02 01.c
2     A first program in C */
3  #include <stdio.h>
4
5  int main()
6  {
7     printf( "Welcome to CSE 103!\n" );
8
9     return 0;
10 }
```

Welcome to CSE 103!

- Comments
 - Text surrounded by `/*` and `*/` is ignored by computer
 - Used to describe program
- **`#include <stdio.h>`**
 - Preprocessor directive - tells computer to load contents of a certain file
 - **`<stdio.h>`** allows standard input/output operations

A Simple C Program: Printing a Line of Text (II)

- **int main()**
 - C programs contain one or more functions, exactly one of which must be **main**
 - Parenthesis used to indicate a function
 - **int** means that main "returns" an integer value
 - Braces indicate a block
 - The bodies of all functions must be contained in braces

A Simple C Program: Printing a Line of Text (III)

- `printf( "Welcome to C!\n" );`
 - Instructs computer to perform an action
 - Specifically, prints string of characters within quotes
 - Entire line called a statement
 - All statements must end with a semicolon
 - `\` - escape character
 - Indicates that `printf` should do something out of the ordinary
 - `\n` is the newline character

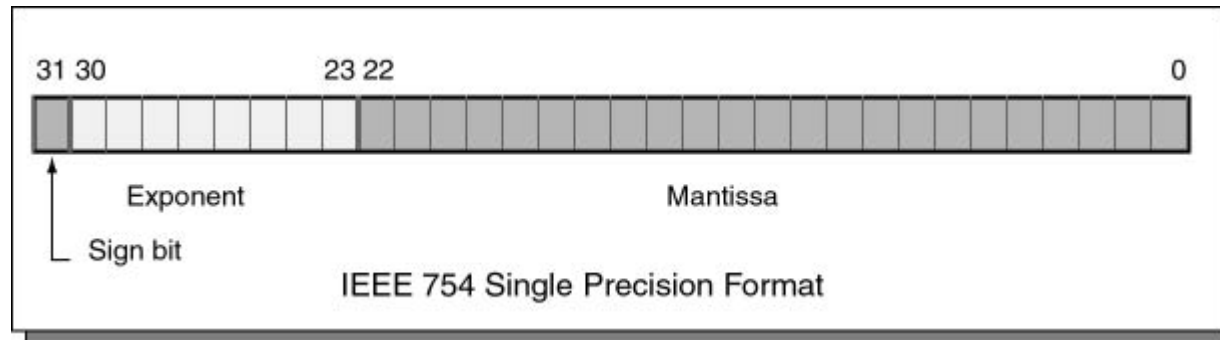
A Simple C Program: Printing a Line of Text (IV)

- **return 0;**
 - A way to exit a function
 - **return 0**, in this case, means that the program terminated normally
- Right brace **}**
 - Indicates end of **main** has been reached

Basic Data types

Type	Example	Size	Range	Format Code
char	A B C..Z a b c..z @ \$ % *	8 bits, 1 byte	<i>-128 to +127</i>	"%c"
int	32, 3, 4 ,5 -4, -5, -6	32 bits, 4 bytes	<i>-2,14,74,83,648 to +2,14,74,83,647</i>	"%d" "%i"
float	3.4, 5.25, -2.3, -6.7	32 bits, 4 bytes	<i>3.4E-38 to 3.4E+38</i>	"%f"
double	3.4, 5.25, -2.3, -6.7	64 bits, 8 bytes	<i>1.7E-308 to 1.7E+308</i>	"%lf"

Float format



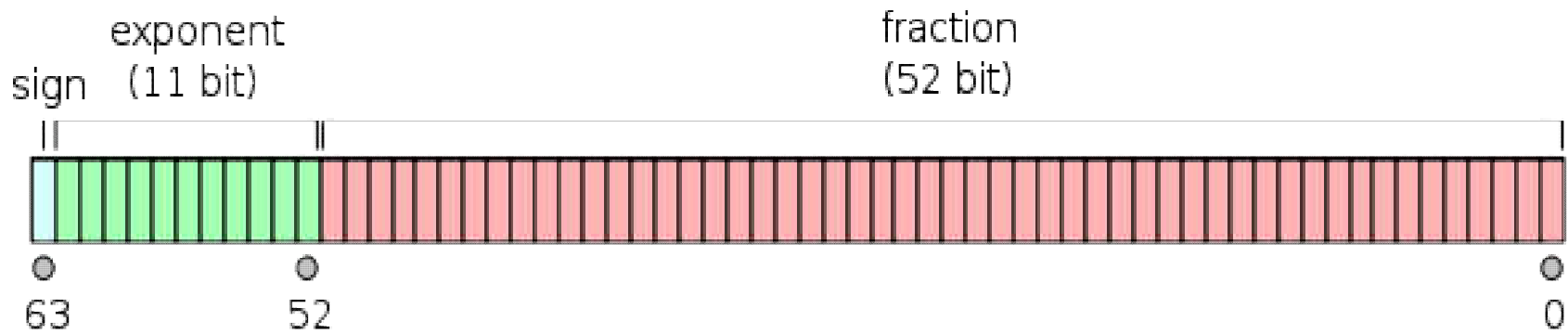
Single precision floating point:

Sign bit: 1

Exponent: 8 bits

Mantissa: 23 bits

Double format



Double precision floating point:

Sign bit: 1

Exponent: 11 bits

Mantissa: 52 bits

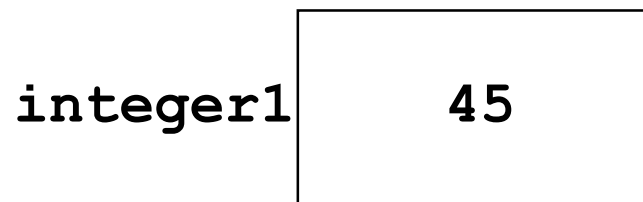
ASCII Values

TABLE 3
ASCII CHARACTER CODES (DECIMAL)

0	Ctrl-@	32	Space	64	@	96	`
1	Ctrl-A	33	!	65	A	97	a
2	Ctrl-B	34	"	66	B	98	b
3	Ctrl-C	35	#	67	C	99	c
4	Ctrl-D	36	\$	68	D	100	d
5	Ctrl-E	37	%	69	E	101	e
6	Ctrl-F	38	&	70	F	102	f
7	Ctrl-G	39	'	71	G	103	g
8	Backspace	40	(	72	H	104	h
9	Tab	41	)	73	I	105	i
10	Ctrl-J	42	*	74	J	106	j
11	Ctrl-K	43	+	75	K	107	k
12	Ctrl-L	44	,	76	L	108	l
13	Return	45	-	77	M	109	m
14	Ctrl-N	46	.	78	N	110	n
15	Ctrl-O	47	/	79	O	111	o
16	Ctrl-P	48	0	80	P	112	p
17	Ctrl-Q	49	1	81	Q	113	q
18	Ctrl-R	50	2	82	R	114	r
19	Ctrl-S	51	3	83	S	115	s
20	Ctrl-T	52	4	84	T	116	t
21	Ctrl-U	53	5	85	U	117	u
22	Ctrl-V	54	6	86	V	118	v
23	Ctrl-W	55	7	87	W	119	w
24	Ctrl-X	56	8	88	X	120	x
25	Ctrl-Y	57	9	89	Y	121	y
26	Ctrl-Z	58	:	90	Z	122	z
27	Escape	59	;	91	[	123	{
28	Ctrl-\	60	<	92	\	124	
29	Ctrl-]	61	=	93	]	125	}
30	Ctrl-^	62	>	94	^	126	~
31	Ctrl-_	63	?	95	_	127	Delete

Memory Concepts

- Variables
 - Variable names correspond to *locations* in the computer's memory.
 - Every variable has a *name*, a *type*, a *size* and a *value*.
 - Whenever a new value is placed into a variable (through **scanf**, for example), it replaces (and destroys) previous value
 - Reading variables from memory does not change them



Using Variables: Output

```
int a, b, c;
```

```
a = 3;
```

```
b = 8;
```

```
c = a + b;
```

```
printf(“%d” , c);
```

Output:

11

Suppose we want.....

```
int a, b, c;
```

```
a = 3;
```

```
b = 8;
```

```
c = a + b;
```

```
printf(“%d” , c);
```

Output:

The value of c: 11

You can do.....

```
int a, b, c;
```

```
a = 3;
```

```
b = 8;
```

```
c = a + b;
```

```
printf("The value of c:");
```

```
printf("%d" , c);
```

Output:

The value of c: 11

printf("The value of c: %d", c)

Or....

```
int a, b, c;
```

```
a = 3;
```

```
b = 8;
```

```
c = a + b;
```

```
printf("The value of c: %d",c);
```

Output:

The value of c: 11

Using Variables: Output

Format specifier

Variable



The diagram shows the `printf` function syntax: `printf(type, what);`. The word `printf` is in red. The parameter `type` is in purple, and the parameter `what` is in green. A red arrow points from the text 'Format specifier' to the `type` parameter. Another red arrow points from the text 'Variable' to the `what` parameter. The entire code snippet is enclosed in a red rectangular border.

Type	Format Code
char	"%c"
int	"%d" "%i"
float	"%f"
double	"%lf"

Using Variables: Output

```
int a, b, c;
```

```
a = 3;
```

```
b = 8;
```

```
c = a + b;
```

```
printf("%d", c);
```

```
char a = 90;
```

```
printf("%c", a);
```

```
float a = 8.958;
```

```
printf("%f", a);
```

```
double a = -9.8;
```

```
printf("%lf", a);
```

```
char a = 'Z';
```

```
printf("%c", a);
```

```
char a = 'Z';
```

```
printf("%d", a);
```

Data input

Format specifier Variable with & sign



```
scanf(what, where);
```

```
int a;  
  
scanf("%i", &a);
```

or

```
scanf("%d", &a);
```

```
float b;  
  
scanf("%f", &b);
```

```
double c;  
  
scanf("%lf", &c);
```

```
char d;  
  
scanf("%c", &d);
```

Little QUIZ (What is the output if user enters 0)

```
int a;  
  
scanf("%i", &a);  
  
printf("%d", a);
```

```
char a;  
  
scanf("%c", &a);  
  
printf("%d", a);
```

Naming convention of a variable

- Capital letters A-Z, lowercase letters a-z, digits 0-9, and the underscore character
- First character must be a letter or underscore
- Usually only the first 31 characters are significant
- There can be no embedded blanks
- Keywords cannot be used as identifiers
- Identifiers are case sensitive

Key words in C

Keywords			
auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Same valid/invalid variable names

-  • First_tag
-  • char
-  • Price\$
-  • group one
-  • average_number
-  • int_
-  • 8boys

Declaring variables

```
1  /* Fig. 2.1: fig02 01.c
2     A first program in C */
3  #include <stdio.h>
4
5  int main()
6  {
7     int interestrate;
8     char ch;
9     return 0;
10 }
```

Assigning values

- Two ways:

- Using **scanf**

```
int c;  
scanf ("%d", &c) ;
```

- Using **assignment operator**

```
int c;  
c = 10;
```

Arithmetic operators on int and char

- Arithmetic operators:

C operation	Arithmetic operator	Algebraic expression	C expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	bm	<code>b * m</code>
Division	/	x / y	<code>x / y</code>
Modulus	%	$r \bmod s$	<code>r % s</code>

- Rules of operator precedence:

Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. If the parentheses are nested, the expression in the innermost pair is evaluated first. If there are several pairs of parentheses “on the same level” (i.e., not nested), they are evaluated left to right.
*, /, or %	Multiplication Division Modulus	Evaluated second. If there are several, they are evaluated left to right.
+ or -	Addition Subtraction	Evaluated last. If there are several, they are evaluated left to right.

Arithmetic (II)

- Arithmetic calculations are used in most programs
 - Use `*` for multiplication and `/` for division
 - Integer division truncates remainder
 - `7 / 5` evaluates to `1`
 - Modulus operator returns the remainder
 - `7 % 5` evaluates to `2`
- Operator precedence
 - Some arithmetic operators act before others (i.e., multiplication before addition)
 - Use parenthesis when needed
 - Example: Find the average of three variables `a`, `b` and `c`
 - Do not use: `a + b + c / 3`
 - Use: `(a + b + c) / 3`

Operator Precedence examples

- Find the values of the followings:

$$5+6*7-8/7 = 46$$

$$5*4/3 = 6$$

$$5/(6+7) - 8\%8 = 0$$

Working with characters

- Take a character as input and print ASCII value.
- Convert uppercase to lower case and vice versa
- Write down a program that will take a capital letter as input and will print n -th letter starting from the letter given as input (**wrap around**). Assume that n is a nonnegative integer less than or equal to 25. Prompt the user to know the value of n . (**You can not perform condition checking to solve this problem**).


```
main( ){  
    int npos,cpos,n;  
    char ch,nch;  
    scanf("%c%d",&ch,&n);  
    cpos = ch - 'A';  
    npos = (cpos +n)%26;  
    nch = npos + 'A';  
    printf("%c",nch);  
}
```

Working with int...examples

- To extract digits of a number:
 - Given a 4 digit number, can you reverse it?
 - Modulus (%) and division (/) operators are good enough.
- Interchange content of two variables:
 - ☐ Using one additional variable
 - ☐ Using no additional variable
- Time difference of two cities.
 - Dhaka: 11:20
 - Mumbai: 10:50

Military time to standard time

Name _____

Date _____

MILITARY TIME CHART 2



Military Time	Standard Time
00:00	12:00 midnight
01:00	1:00am
02:00	2:00am
03:00	3:00am
04:00	4:00am
05:00	5:00am
06:00	6:00am
07:00	7:00am
08:00	8:00am
09:00	9:00am
10:00	10:00am
11:00	11:00am
12:00	12:00 midday
13:00	1:00pm
14:00	2:00pm
15:00	3:00pm
16:00	4:00pm
17:00	5:00pm
18:00	6:00pm
19:00	7:00pm
20:00	8:00pm
21:00	9:00pm
22:00	10:00pm
23:00	11:00pm
00:00	12:00 midnight



Free Math Sheets, Math Games and Math Help

MATH-SALAMANDERS.COM

Increment and Decrement Operators

- Increment operator (**++**) can be used instead of **c=c+1**
- Decrement operator (**--**) can be used instead of **c=c-1**.
- Preincrement
 - Operator is used before the variable (**++c** or **--c**)
 - Variable is changed, then the expression it is in is evaluated
- Postincrement
 - Operator is used after the variable (**c++** or **c--**)
 - Expression executes, then the variable is changed

Increment and Decrement Operators (II)

- When variable not in an expression
 - Preincrementing and postincrementing have the same effect.

```
c = 20 ;  
++c ;  
printf ("%d" , c) ;
```

and

```
c = 20 ;  
c++ ;  
printf ("%d" , c) ;
```

have the same effect.

both will print 21;

Increment and Decrement Operators (III)

- When variable in an expression
 - Pre-incrementing and post-incrementing DOES NOT have the same effect.
 - Preincrement updates the variable first then evaluates expression
 - Postincrement evaluates the expression first then updates the variable

```
c = 5;
```

```
printf( "%d", ++c);           Prints 6
```

```
printf( "%d", c++);          Prints 5
```

In either case, **c** now has the value of 6

Little Quiz for you (what is the output)

```
int a, b, c;  
b = 10;  
c = 20;  
a = b+++--c;  
printf("%d %d %d", a, b, c);
```

- (a) Compilation error
- (b) 30 10 20
- (c) 30 11 19
- (d) 29 11 19
- (e) 29 10 20

OK

Assignment Operators (shorthand notations)

- Assignment operators abbreviate assignment expressions

`c = c + 3;`

can be abbreviated as `c += 3;` using the addition assignment operator

- Statements of the form

variable = variable operator expression;

can be rewritten as

variable operator = expression;

- Examples of other assignment operators:

`d -= 4` `(d = d - 4)`

`e *= 5` `(e = e * 5)`

`f /= 3` `(f = f / 3)`

`g %= 9` `(g = g % 9)`

Arithmetic Operators for float or double

- Arithmetic operators:

C operation	Arithmetic operator	Algebraic expression	C expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	bm	<code>b * m</code>
Division	/	x / y	<code>x / y</code>

Integer division vs Fractional division

Example: Integer division:
 $5/3 = 1$

Example: Fractional division:
 $5.0/3.0=1.667$

What about: $5.0/3$ or $5/3.0$?

Fractional division!!

Working with fractions: Example 1

Sonali Bank annually provides interests at a certain rate to all its clients having a savings account with the bank. Write down a program that will take initial balance, and annual interest rates and will determine and print:

- (1) Balance after one year
- (2) Balance after two years
- (3) Balance after n years where n will also be input to your program.

```
b = 1,00,000
r = 10%
interest1 = 10,000
b1 = 1,00,000 + 10,000
    = 1,10,000
```

```
b1 = 1,10,000
r = 10%
interest2 = 11,000
b2 = 1,10,000 + 11,000
    = 1,21,000
```

Type conversion

- Lower to higher auto-conversion (called auto-casting)

```
int x = 9;  
float y = x; //OK no warning no error
```

- Higher to lower still auto-casting but generates warning

```
float x = 9.5;  
int y = x; //OK but generates warning but no error  
int y = (int) x // No warning called casting
```

- Work out the followings:

```
float x = 5/3;  
int y = 5/3;
```

```
x = 1.0 y = 1
```

```
float x = 5.0/3;  
int y = 5.0/3;
```

```
x = 1.6667 y = 1
```

Type conversion (example)

Floor(x)	$\lfloor x \rfloor$	: The largest integer not exceeding x
Ceil(x)	$\lceil x \rceil$	: The smallest integer not less than x
Round(x)	$\text{round}(x)$	: The nearest integer (in case of tie take greater one)

According to the above definition when $x = 2.3$,
 $\text{floor}(x) = 2$, $\text{ceil}(x) = 3$ and $\text{round}(x) = 2$

Write down a program that will take a positive fractional number as input and will print its floor, ceil and round.

Problem Solving Methodology

- 1. State the problem clearly**
- 2. Describe the input/output information**
- 3. Work the problem by hand, give example**
- 4. Develop a solution (Algorithm Development)
and Convert it to a program (C program)**
- 5. Test the solution with a variety of data**

Working with fractions: Example 2

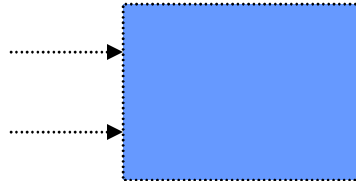
1. Problem statement

Compute the straight line
distance between two points in a plane

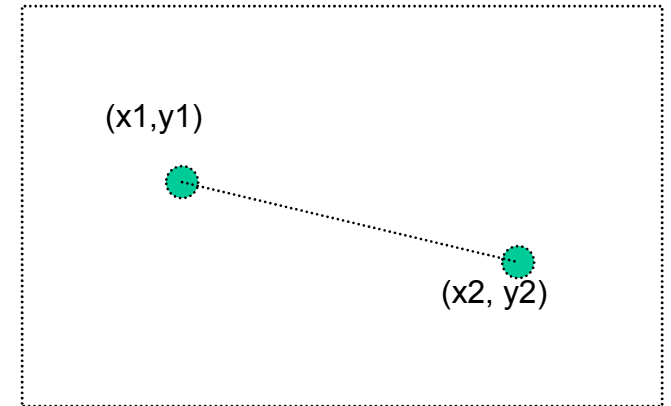
2. Input/output description

Point 1 (x_1, y_1)

Point 2 (x_2, y_2)



Distance between two points (distance)



Example 2 (cont'd)

3. Hand example

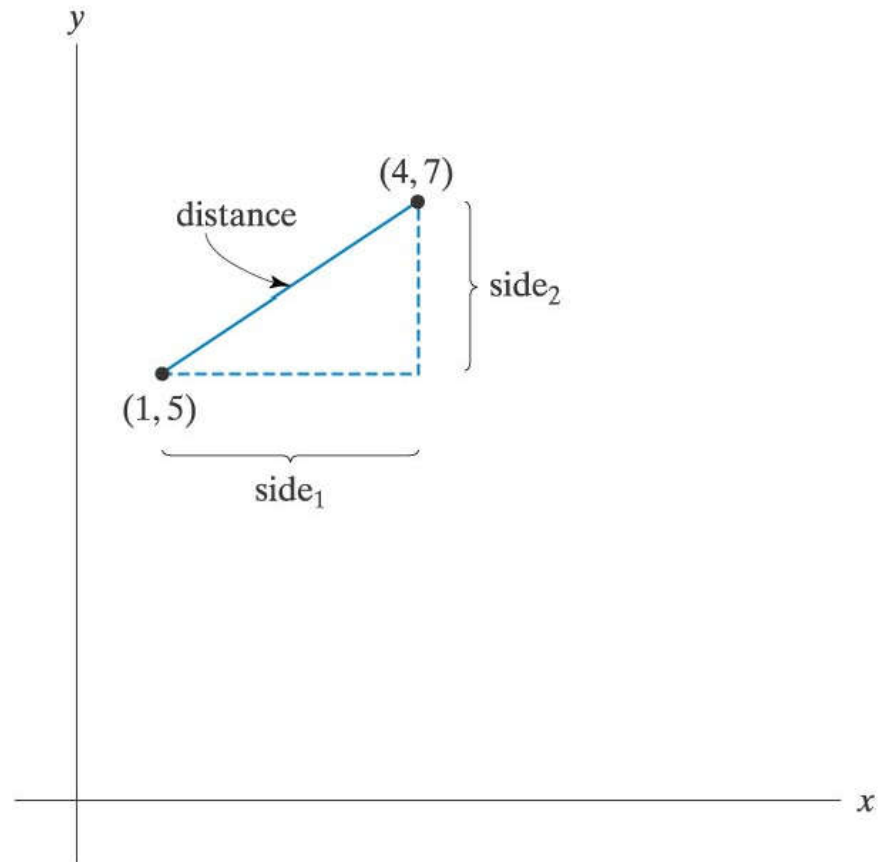
$$\text{side1} = 4 - 1 = 3$$

$$\text{side2} = 7 - 5 = 2$$

$$\text{distance} = \sqrt{\text{side1}^2 + \text{side2}^2}$$

$$\text{distance} = \sqrt{3^2 + 2^2}$$

$$\text{distance} = \sqrt{13} = 3.61$$



Example 2 (cont'd)

4. Algorithm development and coding

- a. Generalize the hand solution and list/outline the necessary operations step-by-step
 - 1) Give specific values for point1 (x1, y1) and point2 (x2, y2)
 - 2) Compute side1=x2-x1 and side2=y2-y1
 - 3) Compute $\text{distance} = \sqrt{\text{side1}^2 + \text{side2}^2}$
 - 4) Print distance
- b. Convert the above outlined solution to a program using any language you want (see next slide for C imp.)

Example 2 (cont'd)

```
/*-----*/
/* Program chapter1_1 */
/* */
/* This program computes the */
/* distance between two points. */
#include <stdio.h>
#include <math.h>

int main(void)
{
    /* Declare and initialize variables. */
    double x1=1, y1=5, x2=4, y2=7,
           side_1, side_2, distance;

    /* Compute sides of a right triangle. */
    side_1 = x2 - x1;
    side_2 = y2 - y1;
    distance = sqrt(side_1*side_1 + side_2*side_2);

    /* Print distance. */
    printf("The distance between the two points is "
           "%5.2f \n",distance);

    /* Exit program. */
    return 0;
}
/*-----*/
```

Example 2 (cont'd)

5. Testing

- After compiling your program, run it and see if it gives the correct result.
- Your program should print out
The distance between two points is 3.61
- If not, what will you do?

Modification to Example 2

How will you find the distance between two other points (2,5) and (10,8)?

```
/*-----*/
/*  Program chapter1_1                               */
/*                                                    */
/*  This program computes the                         */
/*  distance between two points.                     */
/*                                                    */
#include <stdio.h>
#include <math.h>

int main(void)
{
    /*  Declare and initialize variables.  */
    double x1=1, y1=5, x2=4, y2=7, x1=2, y1=5, x2=10, y2=8,
           side_1, side_2, distance;

    /*  Compute sides of a right triangle.  */
    side_1 = x2 - x1;
    side_2 = y2 - y1;
    distance = sqrt(side_1*side_1 + side_2*side_2);

    /*  Print distance.  */
    printf("The distance between the two points is "
           "%5.2f \n", distance);

    /*  Exit program.  */
    return 0;
}
/*-----*/
```